Assignment #10 - Multi-OS EC2 Configuration with Ansible

Overview

This assignment demonstrates the provisioning and configuration of a hybrid AWS EC2 infrastructure using Terraform and Ansible. The setup includes six EC2 instances (3 Ubuntu and 3 Amazon Linux), all tagged accordingly, along with one dedicated EC2 instance as the Ansible Controller. The infrastructure was configured using a Terraform script, and Ansible was used to install Docker, validate its version, and print disk usage details on each instance.

Step 1: Terraform EC2 Provisioning

Using Terraform, seven EC2 instances were provisioned:

- 3 Ubuntu Instances
- 3 Amazon Linux Instances
- 1 Ansible Controller Instance

Each EC2 instance was tagged with the appropriate OS type. Terraform outputs confirmed the instance IPs and ensured they passed health checks.

```
Apply complete! Resources: 9 added, 0 changed, 0 destroyed.

Outputs:

amazon_linux_instance_ips = [
  "3.91.33.206",
  "54.210.62.68",
  "54.242.80.213",
]
ansible_controller_ip = "3.84.104.197"
ubuntu_instance_ips = [
  "54.146.160.42",
  "44.220.163.205",
  "3.88.176.1",
]
```

Step 2: SSH Key Setup

The SSH private key was added to the SSH agent to allow Ansible to connect to the EC2 instances securely. This is crucial for remote provisioning via Ansible.

Assignment #10 - Multi-OS EC2 Configuration with Ansible

Amazon-Linux-Instance-1	i-0074cec3dcab768f1	Running	t2.micro	2/2 checks passec	View alarms +	us-east-1
Ubuntu-Instance-3	i-00858112046f6be2d	Running	t2.micro	2/2 checks passec	View alarms +	us-east-1
Ansible-Controller	i-06bf8442384da1144	Running	t2.micro	2/2 checks passec	View alarms +	us-east-1
Ubuntu-Instance-1	i-0469f69a7b02318f6	Running	t2.micro	2/2 checks passec	View alarms +	us-east-1
Amazon-Linux-Instance-3	i-0fb3b82a5ec8cc342	Running	t2.micro	2/2 checks passec	View alarms +	us-east-1
Amazon-Linux-Instance-2	i-0d151720336bbf602	Running	t2.micro	2/2 checks passec	View alarms +	us-east-1
Ubuntu-Instance-2	i-0eed9240185167ea2	Running	t2.micro	2/2 checks passec	View alarms +	us-east-1

Step 3: Packer AMI Creation

Two separate AMIs were built using Packer for Ubuntu and Amazon Linux, both of which included Docker pre-installed.

```
(.venv) sureshrajaselvadurai@Sureshs-MacBook-Air terraform-aws-setup % eval "$(ssh-agent -s)"
ssh-add /Users/sureshrajaselvadurai/.ssh/devops_a10
Agent pid 1791
Enter passphrase for /Users/sureshrajaselvadurai/.ssh/devops_a10:
Identity added: /Users/sureshrajaselvadurai/.ssh/devops_a10 (sureshrajaselvadurai@Sureshs-MacBook-Air.local)
(.venv) sureshrajaselvadurai@Sureshs-MacBook-Air packer-ami % packer build amazon-linux-docker.json
amazon-ebs.amazon-linux: output will be in this color.

==> amazon-ebs.amazon-linux: Prevalidating any provided VPC information
==> amazon-ebs.amazon-linux: Prevalidating AMI Name: ami-linux-docker-a10.1-1743433419
    amazon-ebs.amazon-linux: Found Image ID: ami-072e42fd77921edac
==> amazon-ebs.amazon-linux: Creating temporary keypair: packer_67eaaecb-2d52-5324-3e75-6646e82a9f6f
==> amazon-ebs.amazon-linux: Creating temporary security group for this instance: packer_67eaaecd-00c5-2594-47b2-af76ba9afe14
==> amazon-ebs.amazon-linux: Authorizing access to port 22 from [0.0.0.0/0] in the temporary security groups...
==> amazon-ebs.amazon-linux: Launching a source AWS instance...
    amazon-ebs.amazon-linux: Instance ID: i-0741e825bb3cebbe8
==> amazon-ebs.amazon-linux: Waiting for instance (i-0741e825bb3cebbe8) to become ready...
```

Step 4: Ansible Playbook Execution

An Ansible playbook was created targeting all six EC2 instances. The playbook executed the following tasks:

1. Updated and upgraded packages (Ubuntu and Amazon Linux).
2. Started and enabled Docker.
3. Verified Docker version.
4. Printed disk usage on each instance.

```
(.venv) sureshrajaselvadurai@Sureshs-MacBook-Air packer-ami % packer build ubuntu-docker.json
amazon-ebs.ubuntu: output will be in this color.

==> amazon-ebs.ubuntu: Prevalidating any provided VPC information
==> amazon-ebs.ubuntu: Prevalidating AMI Name: ubuntu-docker-a10.1-1743434067
    amazon-ebs.ubuntu: Found Image ID: ami-0cd843ccb7d30d8d9
==> amazon-ebs.ubuntu: Creating temporary keypair: packer_67eab153-f5f4-ae31-f4c7-9d441f87067a
==> amazon-ebs.ubuntu: Creating temporary security group for this instance: packer_67eab156-6480-5f1e-3866-19c61f32f6e8
==> amazon-ebs.ubuntu: Authorizing access to port 22 from [0.0.0.0/0] in the temporary security groups...
==> amazon-ebs.ubuntu: Launching a source AWS instance...
    amazon-ebs.ubuntu: Instance ID: i-00047cfee8dda88e8
==> amazon-ebs.ubuntu: Waiting for instance (i-00047cfee8dda88e8) to become ready...
```

Assignment #10 - Multi-OS EC2 Configuration with Ansible

```
TASK [Start and enable Docker] ************************************************************************************************************
ok: [18.207.216.138]
ok: [54.237.197.8]
ok: [3.91.149.97]
ok: [54.87.37.229]

TASK [Check Docker version] ***************************************************************************************************************
ok: [3.91.149.97]
ok: [54.87.37.229]
ok: [54.237.197.8]
ok: [18.207.216.138]

TASK [Print Docker version] ***************************************************************************************************************
ok: [18.207.216.138] => {
    "msg": "Docker version 26.1.3, build 26.1.3-0ubuntu1~22.04.1+esm1"
}
ok: [54.237.197.8] => {
    "msg": "Docker version 26.1.3, build 26.1.3-0ubuntu1~22.04.1+esm1"
}
ok: [54.87.37.229] => {
    "msg": "Docker version 26.1.3, build 26.1.3-0ubuntu1~22.04.1+esm1"
}
ok: [3.91.149.97] => {
    "msg": "Docker version 26.1.3, build 26.1.3-0ubuntu1~22.04.1+esm1"
}

TASK [Print disk usage] *******************************************************************************************************************
ok: [18.207.216.138] => {
    "disk_usage.stdout_lines": [
        "Filesystem      Size  Used Avail Use% Mounted on",
        "/dev/root        20G  2.1G   18G  11% /",
        "tmpfs           479M     0  479M   0% /dev/shm",
        "tmpfs           192M  904K  191M   1% /run",
        "tmpfs           5.0M     0  5.0M   0% /run/lock",
        "/dev/xvda15     105M  6.1M   99M   6% /boot/efi",
        "tmpfs            96M  4.0K   96M   1% /run/user/1000"
    ]
}
ok: [54.237.197.8] => {
    "disk_usage.stdout_lines": [
        "Filesystem      Size  Used Avail Use% Mounted on",
        "/dev/root        20G  2.1G   18G  11% /",
        "tmpfs           479M     0  479M   0% /dev/shm",
        "tmpfs           192M  904K  191M   1% /run",
        "tmpfs           5.0M     0  5.0M   0% /run/lock",
        "/dev/xvda15     105M  6.1M   99M   6% /boot/efi",
        "tmpfs            96M  4.0K   96M   1% /run/user/1000"
    ]
ok: [54.87.37.229] => {
    "disk_usage.stdout_lines": [
        "Filesystem      Size  Used Avail Use% Mounted on",
        "/dev/root        20G  2.1G   18G  11% /",
        "tmpfs           479M     0  479M   0% /dev/shm",
        "tmpfs           192M  904K  191M   1% /run",
        "tmpfs           5.0M     0  5.0M   0% /run/lock",
        "/dev/xvda15     105M  6.1M   99M   6% /boot/efi",
        "tmpfs            96M  4.0K   96M   1% /run/user/1000"
    ]
}
ok: [3.91.149.97] => {
    "disk_usage.stdout_lines": [
        "Filesystem      Size  Used Avail Use% Mounted on",
        "/dev/root        20G  2.1G   18G  11% /",
        "tmpfs           479M     0  479M   0% /dev/shm",
        "tmpfs           192M  904K  191M   1% /run",
        "tmpfs           5.0M     0  5.0M   0% /run/lock",
        "/dev/xvda15     105M  6.1M   99M   6% /boot/efi",
        "tmpfs            96M  4.0K   96M   1% /run/user/1000"
    ]
}
```

Assignment #10 - Multi-OS EC2 Configuration with Ansible

Conclusion

This setup confirms successful multi-OS provisioning using Terraform and cross-platform configuration using Ansible. Docker was installed and validated, and disk usage reports were retrieved from all instances. The code and instructions are available in the specified GitHub repository under the assignment10 branch.